Lycée Duhamel du Monceau. Niveau : Première.		
	Numérique et Sciences Informatiques	
	Premiers pas avec Python	

Programmer un ordinateur, c'est quoi ?
Programmer, c'est créer des programmes (suite d'ordres donnés à l'ordinateur) ! Un ordinateur sans programme ne sait rien faire. Il existe différents langages qui permettent de programmer un ordinateur, mais le seul directement utilisable par le processeur est le langage machine (suite de 1 et de 0), aussi appelé binaire. Aujourd'hui (presque) plus personne ne programme en binaire (trop compliqué).
Les informaticiens utilisent des instructions (mots souvent en anglais) en lieu et place de la suite de 0 et de 1. Ces instructions, une fois écrites par le programmeur, sont « traduites » en langage machine. Un programme spécialisé assure cette traduction. Ce système de traduction s'appellera interpréteur ou bien compilateur, suivant la méthode utilisée pour effectuer la traduction.
Il existe 2 grandes familles de langages de programmation :

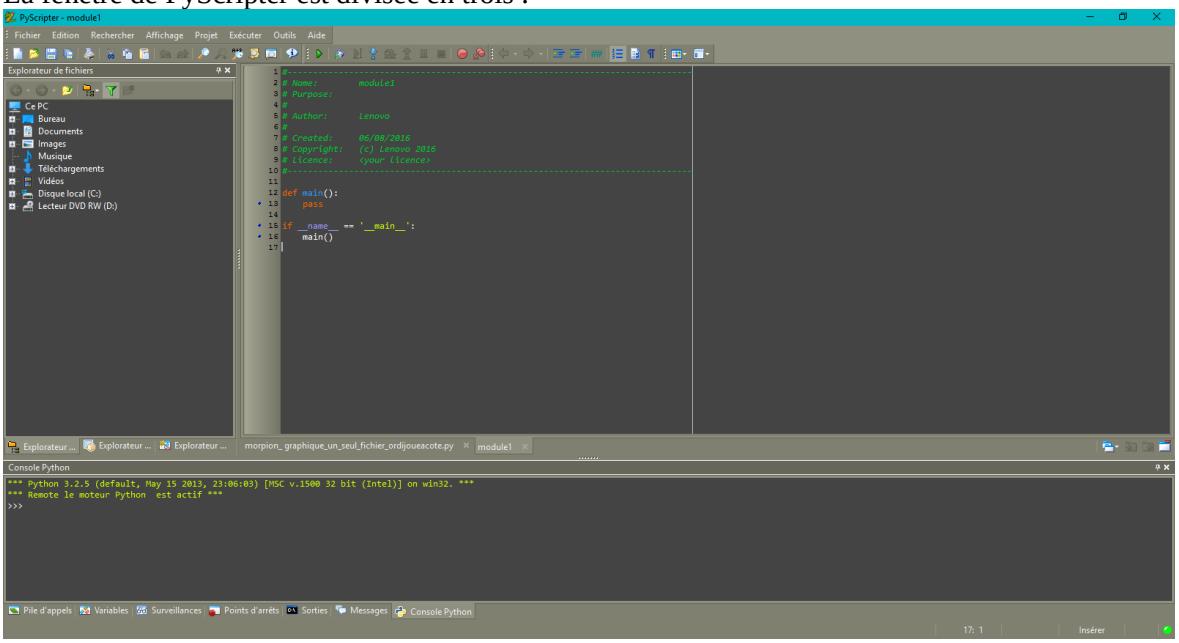
- Les langages de bas niveau sont très complexes à utiliser, car très éloignés du langage naturel, on dit que ce sont des langages « proches de la machine », en contrepartie ils permettent de faire des programmes très rapides à l'exécution. L'assembleur est le langage de bas niveau. Certains "morceaux" de programmes sont écrits en assembleur encore aujourd'hui.
- Les langages de haut niveau sont eux plus « faciles » à utiliser, car plus proches du langage naturel (exemple : si a=3 alors b=c). Exemples de langages de haut niveau : C, C++, java, Qbasic...

I) <u>AVANT TOUT</u> :

Lancer EduPython. On pourra travailler avec le IDLE (Environnement de Développement Intégré au Langage Python) fourni avec Python, mais il vaut mieux utiliser un éditeur de code. Il en existe une multitude, avec leurs qualités et leurs défauts (PyScripter, Spyder, PyCharm, Visual Studio Code, …).
Avec la distribution EduPython, c'est PyScripter qui a été choisi.
Dans « Mes documents », créer un dossier NSI\Python, dans lequel tous vos fichiers seront stockés.

II) <u>PRISE EN MAIN</u> :

Il existe plusieurs Environnements de Développement Intégré (EDI, ou IDLE en anglais). Nous utiliserons principalement PyScripter (mais libre à chacun de choisir l'IDLE qui lui convient le mieux).
Python et Pyscripter peuvent être installés :
• Sur un ordinateur classiquement (ainsi vos fichiers .py s'ouvriront avec l'éditeur fourni PyScripter) ;
• Ou sur une clé USB ou un lecteur réseau sans qu'il soit nécessaire de disposer de droits d'administrateur.
La fenêtre de PyScripter est divisée en trois :



• en haut à gauche, trois onglets possibles :
– l'explorateur de fichier pour parcourir l'arborescence des répertoires et choisir les fichiers à ouvrir ;
– l'explorateur de projet utile si on développe des programmes plus importants nécessitant plusieurs scripts ;
– l'explorateur de code qui permet de retrouver rapidement les fonctions et variables créées dans un script (surtout utile quand le script est long...)
• en haut à droite, l'éditeur dans lequel on tapera les programmes ;
• en bas, à nouveau plusieurs onglets possibles. Les plus utiles sont :
– la Console Python. Cette console permet de taper directement des instructions Python (souvent pour faire des tests, ou de petits calculs), et c'est aussi dans cet onglet que l'on pourra visualiser l'exécution des programmes tapés dans l'éditeur.
– l'onglet variables qui nous servira à visualiser la valeur des variables utilisées, principalement pour debugger les programmes que l'on tapera.
Toutes ces fenêtres et onglets peuvent être fermés ou déplacés par un glisser-déposer à la souris, ou en allant dans le menu affichage.

1°) La console :

Dans la console, on peut taper directement des instructions Python qui s'exécutent immédiatement (à condition d'appuyer sur <Entrée> en fin de ligne). La division s'obtient avec /. On peut faire des divisions entières avec //, et calculer des modulo avec %. La puissance s'obtient par ** (attention de ne pas confondre avec ^ qui calcule un ou exclusif). Le résultat du dernier calcul est rappelé par _ (tiret du 8) :

```
>>> 3 + 4
7
>>> 5 * 14
70
>>> _ +5
75
>>> 11 / 4
2.75
>>> 11 // 4
2
>>> 11 % 4
3
>>> 2 ** 10
1024
>>> 2 ^ 10
8
```

On peut ajouter des commentaires en commençant par un # :

```
>>> 3 * 5 # Un calcul très compliqué!!!
15
```

On peut aussi affecter des variables avec =, afficher du texte avec print :

```
>>> a = 3
>>> b = 5
>>> a + b
8
>>> a = a - 1
>>> a = b
>>> b = a
>>> print('Salut, monde!')
Salut, monde!
```

Exercice 1 : Après avoir tapé les lignes ci-dessus, quelles sont les valeurs de a et b ? (Rappel : un des onglets s'appelle Explorateur de variables...). Quelle suite d'instructions faudrait-il taper pour échanger les valeurs de deux variables ?

2°) L'éditeur :

L'éditeur sert à taper des programmes (en Python, on parle aussi de scripts) plus complets, que l'on pourra sauvegarder dans des fichiers dont l'extension est .py

Exercice 2 : Dans l'éditeur, taper le programme suivant :

```
print("Bonjour")
x = 42
print(x)
```

Observer la coloration syntaxique, l'ajout automatique de parenthèse fermante, la coloration des paires de parenthèses correspondantes.
Sauvegarder ce programme dans un répertoire adapté (rappelez-vous les répertoires créés au début de ce TP...).
Essayer de donner un nom explicite à vos dossiers et fichiers, en évitant autant que possible les lettres accentuées et les espaces.
Exécuter ensuite ce programme (soit par le menu Exécution, soit en cliquant sur le bouton, soit en appuyant sur Ctrl+F9).

III) EXPRESSIONS ET TYPES SIMPLES :

1°) Les différents types de variables :

Une variable est un espace mémoire dans lequel il est possible de stocker une valeur (une donnée).

a) Noms de variables :

Le nom d'une variable s'écrit avec des lettres (non accentuées), des chiffres ou bien l'underscore _
Le nom d'une variable ne doit pas commencer par un chiffre.
En langage Python, l'usage est de ne pas utiliser de lettres majuscules pour nommer les variables (celles-ci sont réservées pour nommer les classes).
Exemple : age, mon_age, temperature1.

A éviter : ~~Age, AGE, MonAge, monAge, Temperature1~~

b) Le type int (integer : nombres entiers) :

Pour affecter (on dit aussi assigner) la valeur 17 à la variable nommée age :

```
>>> age = 17
```

La fonction print() affiche la valeur de la variable :

```
>>> print(age)
17
```

La fonction type() retourne le type de la variable :

```
>>> print(type(age))
<class 'int'>
```

int est le type des nombres entiers.

```
>>> # ceci est un commentaire
>>> age = age + 1    # en plus court : age += 1
>>> print(age)
18
>>> age = age - 3    # en plus court : age -= 3
>>> print(age)
15
>>> age = age*2                # en plus court : age *= 2
>>> print(age)
30
>>> a = 6*3 - 20
>>> print(a)
-2
>>> b = 25
```

```
>>> c = a + 2*b
>>> print(b, c)        # ne pas oublier la virgule
25 48
```

L'opérateur // donne la division entière :

```
>>> tour = 450//360
>>> print(tour)
1
```

L'opérateur % donne le reste de la division (opération modulo) :

```
>>> angle = 450%360
>>> print(angle)
90
```

L'opérateur ** donne la puissance :

```
>>> mo = 2**20
>>> print(mo)
1048576
>>> racine2 = 2**0.5
>>> print(racine2)
1.41421356237
```

c) Le type float (nombres en virgule flottante) :

```
>>> b = 17.0          # le séparateur décimal est un point (et non une virgule)
>>> print(b)
17.0
>>> print(type(b))
<class 'float'>
>>> c = 14.0/3.0
>>> print(c)
4.66666666667
>>> c = 14.0//3.0     # division entière
>>> print(c)
4.0
```

Attention : avec des nombres entiers, l'opérateur / fait une division classique et retourne un type float :

```
>>> c = 14/3
>>> print(c)
4.66666666667
Notation scientifique :
>>> a = -1.784892e4
>>> print(a)
-17848.92
```

Les fonctions mathématiques

Pour utiliser les fonctions mathématiques, il faut commencer par importer le module math :

```
>>> import math
```

La fonction dir() retourne la liste des fonctions et données d'un module :

```
>>> dir(math)
['__doc__', '__name__', '__package__', 'acos', 'acosh', 'asin', 'asinh', 'atan',
'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf',
'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum',
'gamma', 'hypot', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p',
'modf', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'trunc']
```

Pour appeler une fonction d'un module, la syntaxe est la suivante :
module.fonction(arguments)

Pour accéder à une donnée d'un module :
module.data

```
>>> print(math.pi)                # donnée pi du module math (nombre pi)
3.14159265359
>>> print(math.sin(math.pi/4.0)) # fonction sin() du module math (sinus)
0.707106781187
>>> print(math.sqrt(2.0))         # fonction sqrt() du module math (racine carrée)
1.41421356237
>>> print(math.sqrt(-5.0))
Traceback (most recent call last):
   print math.sqrt(-5.0)
ValueError: math domain error
>>> print(math.exp(-3.0))         # fonction exp() du module math (exponentielle)
0.0497870683679
>>> print(math.log(math.e))       # fonction log() du module math (logarithme népérien)
1.0
```

d) Le type str (string : chaîne de caractères) :

```
>>> nom = 'Dupont'                # entre apostrophes
>>> print(nom)
Dupont
>>> print(type(nom))
<class 'str'>
>>> prenom = "Pierre"             # on peut aussi utiliser les guillemets
>>> print(prenom)
Pierre
>>> print(nom, prenom)            # ne pas oublier la virgule
Dupont Pierre
```

La concaténation désigne la mise bout à bout de plusieurs chaînes de caractères.
La concaténation utilise l'opérateur +

```
>>> chaine = nom + prenom    # concaténation de deux chaînes de caractères
>>> print(chaine)
DupontPierre
>>> chaine = prenom + nom    # concaténation de deux chaînes de caractères
>>> print(chaine)
PierreDupont
>>> chaine = prenom + ' ' + nom
>>> print(chaine)
Pierre Dupont
>>> chaine = chaine + ' 18 ans'# en plus court : chaine += ' 18 ans'
>>> print(chaine)
Pierre Dupont 18 ans
La fonction len() retourne la longueur (length) de la chaîne de caractères :
>>> print(len(chaine))
20
```

Indexage et slicing :

```
>>> print(chaine[0])              # premier caractère (indice 0)
P
>>> print(chaine[1])# deuxième caractère (indice 1)
i
>>> print(chaine[1:4])            # slicing
ier
>>> print(chaine[2:])             # slicing
erre Dupont 18 ans
>>> print(chaine[-1])             # dernier caractère (indice -1)
s
>>> print(chaine[-6:])            # slicing
```

```
18 ans
```

En résumé :

```
 +---+---+---+---+---+---+
 | M | u | r | i | e | l |
 +---+---+---+---+---+---+
 0   1   2   3   4   5   6
-6  -5  -4  -3  -2  -1
```

```
>>> chaine = 'Aujourd'hui'
SyntaxError: invalid syntax
>>> chaine  = 'Aujourd\'hui'                    # séquence d'échappement \'
>>> print(chaine)
Aujourd'hui
>>> chaine  = "Aujourd'hui"
>>> print(chaine)
Aujourd'hui
```

La séquence d'échappement \n représente un saut ligne :

```
>>> chaine = 'Première ligne\nDeuxième ligne'
>>> print(chaine)
Première ligne
Deuxième ligne
```

Plus simplement, on peut utiliser les triples guillemets (ou les triples apostrophes) pour encadrer une chaîne définie sur plusieurs lignes :

```
>>> chaine = """Première ligne
Deuxième ligne"""
>>> print(chaine)
Première ligne
Deuxième ligne
```

On ne peut pas mélanger les serviettes et les torchons (ici type str et type int) :

```
>>> chaine = '17.45'
>>> print(type(chaine))
<class 'str'>
>>> chaine = chaine + 2
TypeError: Can't convert 'int' object to str implicitly
```

La fonction float() permet de convertir un type str en type float

```
>>> nombre = float(chaine)
>>> print(nombre)
17.45
>>> print(type(nombre))
<class 'float'>
>>> nombre = nombre + 2                         # en plus court : nombre += 2
>>> print(nombre)
19.45
```

La fonction input() lance une invite de commande (en anglais : prompt) pour saisir une chaîne de caractères.

```
>>> # saisir une chaîne de caractères et valider avec la touche Enter
>>> chaine = input("Entrer un nombre : ")
Entrer un nombre : 14.56
>>> print(chaine)
14.56
>>> print(type(chaine))
<class 'str'>
>>> nombre = float(chaine)     # conversion de type
```

```
>>> print(nombre**2)
211.9936
```

e) Le type list (liste) :

Une liste est une structure de données.
Le premier élément d'une liste possède l'indice (l'index) 0.
Dans une liste, on peut avoir des éléments de plusieurs types.

```
>>> infoperso = ['Pierre', 'Dupont', 17, 1.75, 72.5]
>>> # la liste infoperso contient 5 éléments de types str, str, int, float et float
>>> print(type(infoperso))
<class 'list'>
>>> print(infoperso)
['Pierre', 'Dupont', 17, 1.75, 72.5]
>>> print('Prénom : ', infoperso[0])                    # premier élément (indice 0)
Prénom :  Pierre
>>> print('Age : ', infoperso[2])                       # le troisième élément a l'indice 2
Age :  17
>>> print('Taille : ', infoperso[3])                    # le quatrième élément a l'indice 3
Taille :  1.75
```

La fonction range() crée une liste d'entiers régulièrement espacés :

```
>>> maliste = range(10)
>>> print(list(maliste))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> print(type(maliste))
<class 'range'>
>>> maliste = range(1,10,2)     # range(début,fin non comprise,intervalle)
>>> print(list(maliste))
[1, 3, 5, 7, 9]
>>> print(maliste[2])                    # le troisième élément a l'indice 2
5
```

On peut créer une liste de listes, qui s'apparente à un tableau à 2 dimensions (ligne, colonne) :

```
 0  1  2
10 11 12
20 21 22
>>> maliste = [[0, 1, 2], [10, 11, 12], [20, 21, 22]]
>>> print(maliste[0])
[0, 1, 2]
>>> print(maliste[0][0])
0
>>> print(maliste[2][1])                 # élément à la troisième ligne et deuxième colonne
21
>>> maliste[2][1] = 69                   # nouvelle affectation
>>> print(maliste)
[[0, 1, 2], [10, 11, 12], [20, 69, 22]]
```

f) Le type bool (booléen) :

Deux valeurs sont possibles : True et False

```
>>> choix = True
>>> print(type(choix))
<class 'bool'>
```

Les opérateurs de comparaison :

Opérateur	Signification	Remarques
<	strictement inférieur	
<=	inférieur ou égal	

Opérateur	Signification	Remarques
>	strictement supérieur	
>=	supérieur ou égal	
==	égal	Attention : deux signes ==
!=	différent	

```
>>> b = 10
>>> print(b > 8)
True
>>> print(b == 5)
False
>>> print(b != 10)
False
>>> print(0 <= b <= 20)
True
```

Les opérateurs logiques : and, or, not

```
>>> note = 13.0
>>> mention_ab = note >= 12.0 and note < 14.0 # ou bien : mention_ab = 12.0 <= note < 14.0
>>> print(mention_ab)
True
>>> print(not mention_ab)
False
>>> print(note == 20.0 or note == 0.0)
False
```

L'opérateur in s'utilise avec des chaînes (type str) ou des listes (type list) :

```
>>> chaine = 'Bonsoir'
>>> # la sous-chaîne 'soir' fait-elle partie de la chaîne 'Bonsoir' ?
>>> resultat = 'soir' in chaine
>>> print(resultat)
True
>>> print('b' in chaine)
False
>>> maliste = [4, 8, 15]
>>> # le nombre entier 9 est-il dans la liste ?
>>> print(9 in maliste)
False
>>> print(8 in maliste)
True
>>> print(14 not in maliste)
True
```

g) Le type dict (dictionnaire) :

Un dictionnaire stocke des données sous la forme **clé ⇒ valeur**

Une clé est unique et n'est pas nécessairement un entier (comme c'est le cas de l'indice d'une liste).

```
>>> moyennes = {'math': 12.5, 'anglais': 15.8}        # entre accolades
>>> print(type(moyennes))
<class 'dict'>
>>> print(moyennes['anglais'])              # entre crochets
15.8
>>> moyennes['anglais'] = 14.3                        # nouvelle affectation
>>> print(moyennes)
{'anglais': 14.3, 'math': 12.5}
>>> moyennes['sport'] = 11.0                # nouvelle entrée
>>> print(moyennes)
{'sport': 11.0, 'anglais': 14.3, 'math': 12.5}
```

2°) Retour sur les différents types :

En python, les variables sont typées, autrement dit, elles sont classées selon l'objet qu'elles représentent. Une expression peut ainsi être de type entier, flottant, chaîne de caractères, ...
On obtient le type d'une expression en utilisant la fonction type. Par exemple, taper dans la console :

```
type(42)
type(3.5)
type('Bonjour')
x = 3
type(x)
message = 'Voulez-vous rejouer ?'
type(message)
```

3°) Les entiers :

En python, les entiers sont stockés en précision illimitée, ce qui signifie qu'on peut manipuler des entiers aussi grand que l'on veut. Les opérations sur les entiers sont l'addition (+), la soustraction ou l'opposé (-), la multiplication(*), la division (/), la division entière(//), le modulo(%) et la puissance (**). Les ordres de priorité sont les mêmes qu'en mathématiques.

4°) Les flottants :

Les nombres à virgule flottante, ou plus simplement flottants, sont la représentation informatique des nombres décimaux (voir le cours sur la numération). Les opérations sont les mêmes que sur les entiers.
On peut convertir des expressions d'un type à un autre. Par exemple float(42) donne 42.0. On peut aussi taper directement 42.0, ou même 42. sans écrire le zéro. Réciproquement, int renvoie la troncature.

Exercice 3 : Prévoir le résultat de 8.5 / 2.5, de int(8.5) / int(2.5) et de int(8.5 / 2.5). Vérifier.

Exercice 4 : Que renvoie float(7//2) ?

5°) Les booléens :

Un booléen ne peut prendre que deux valeurs : True et False. Il permet de représenter le résultat d'une expression logique. Par exemple, l'expression x % 2 == 0 sera True(vraie) si x est pair, et False(fausse) sinon.
Les opérations sur les booléens sont la négation (le non logique) not, la conjonction (le et logique) and et la disjonction (le ou logique) or.
On peut aussi tester des inégalités, voir des encadrements. Par exemple taper en mode console :

```
1 < 3 #inégalité stricte
1 <= 3 #inégalité large
1 >= 3
1 + 1 != 3 # non égal
1 <= 3 < 5 #encadrement
1 < 3 < 9 == 3 * 3 <= 10 #voir des choses plus compliquées
```

Exercice 5 : Quelle est la valeur des expressions booléennes suivantes, lorsqu'elle existe ?

```
3 * 3.5 > 10
3. * 7 == 21
3 – 1 >= 1
not (True and False)
not (not True)
```

Vérifier les réponses dans une console.

IV) MES PREMIERS PROGRAMMES :

Vous avez découvert « Hello world ! », on va maintenant passer à la vitesse supérieure.

Exercice 6 :
Affectez les variables temps et distance par les valeurs 6.892 et 19.7.
Calculez et affichez la valeur de la vitesse.
Améliorez l'affichage en imposant un chiffre après le point décimal.

Exercice 7 :
Saisir un nom et un âge en utilisant l'instruction input(). Les afficher.

Enfin, utilisez la « bonne pratique » : recommencez l'exercice en *transtypant* les saisies effectuées, c'est à dire en modifiant le type de variable lors de la saisie. Quel est le type de variable saisie avec input ?

Exercice 8 : Distributeur de billets de banque.

Une banque centrale a mis en circulation n types de billets de banque, dont les valeurs sont notées $v1, v2, \ldots, vn$.
On suppose que ces valeurs sont des entiers strictement positifs. On fait en sorte que ces valeurs forment une suite strictement croissante. Enfin, on suppose $n \geq 1$ et $v1 = 10$. Dans cet exercice, on considère que la suite $v1, v2, \ldots, vn$ est fixée et connue.
Un distributeur de banque dispose de billets de 10, 20, 50, 100, 200 et 500 euros. On a $n = 6$ et $v1, v2, \ldots, v6$
Si s est un entier positif multiple de 10 ou nul, on note $p(s)$ le nombre minimal de billets (de tous types) nécessaires pour former la somme s. Par exemple, dans la zone euro, $p(80)$ est égal à 3, car on peut obtenir 80 euros à l'aide de 3 billets (de 10, 20, et 50 euros) et il est impossible d'obtenir cette somme à l'aide de deux billets seulement.
Le problème du distributeur de billets consiste, étant donné s, à calculer p(s). On cherche d'abord un algorithme simple et efficace capable d'exhiber une manière, pas nécessairement optimale, d'atteindre la somme s.

Écrire un programme qui demande la somme désirée et propose une répartition avec le moins de billets possible.

Nous reviendrons sur cet exemple dans la partie algorithmique.